

Scientific Collaboration Network Analyzer

Database Layer — Progress Report

Prepared for project mentor review

Module: Database Design, Authentication & API Layer

Status: Core database layer complete and verified

Stack: Python · FastAPI · SQLAlchemy · PostgreSQL · Alembic · Docker · JWT

Table of Contents

1. Executive Summary	3
2. Project Objective	3
3. What Has Been Built	3
3.1 Database Schema	3
3.2 Schema Migrations	4
3.3 Sample Data	4
4. Authentication and Security	4
5. API Layer	4
5.1 Read Endpoints	5
5.2 Write Endpoints and Validation	5
6. Entity Relationship Diagram	5
7. Testing and Verification	6
8. Technology Stack	7
9. Scope and Ownership	7
10. Remaining Work	8
11. Conclusion	8

1. Executive Summary

This report documents the work completed on the database layer of the Scientific Collaboration Network Analyzer, a research collaboration management platform designed to help universities, research institutes, and funding organizations track publications, researchers, institutions, projects, conferences, and collaborations. The database layer forms the foundation on which the rest of the application — backend business logic, dashboards, reporting, and the frontend — is being built by other members of the team.

As of this report, the database schema, migrations, authentication system, and a validated set of API endpoints have been designed, implemented, and tested end-to-end against a live PostgreSQL database. Every feature described below has been exercised with real requests and real data, not just written and left unverified.

2. Project Objective

The platform tracks co-authorship, research groups, publications, funding projects, conference participation, citations, and institutional partnerships through a centralized database, and provides visualization of collaboration networks, publication statistics, institutional reports, and researcher profiles without using AI-based analysis. It is intended for use by universities, research institutes, government laboratories, academic publishers, and funding organizations.

3. What Has Been Built

3.1 Database Schema

A relational schema was designed and implemented in PostgreSQL using SQLAlchemy, consisting of fourteen interconnected tables covering every core module of the specification.

Table	Purpose
users	Login accounts, password hashes, roles, verification status
institutions	Universities, research institutes, labs, publishers, funders
researchers	Academic profiles linked one-to-one with a user account
tags / researcher_tags	Reusable skill and research-interest keywords (many-to-many)
publications	Journal papers, conference papers, books, patents, reports
publication_authors	Co-authorship with author order and corresponding-author flag
projects	Funded research projects / grants
project_members	Researchers assigned to a project, with their role
collaborations	Institutional partnerships, optionally tied to a project

Table	Purpose
conferences / conference_participations	Events and researcher participation records
citations	Internal and external citation records
audit_logs	Action history for compliance and security review

Co-authorship, project membership, and conference participation are modeled as full association tables rather than simple many-to-many links, so extra information — such as author order, corresponding-author status, and project role — can be captured accurately.

3.2 Schema Migrations

Alembic is used to manage schema versioning. Every change to the data model is captured as a migration script, allowing the database to evolve safely over time without losing existing data, and giving the team a complete history of structural changes.

3.3 Sample Data

A seeding script populates the database with realistic sample data — institutions, researchers, publications with co-authors, funded projects, conferences, and citation records — so the system can be demonstrated and tested without depending on manual data entry.

4. Authentication and Security

A full authentication system was implemented on top of the database layer, going beyond a simple login form to reflect how real-world systems verify user identity.

- Passwords are never stored in plain text — they are hashed using bcrypt before being written to the database.
- Registration validates not only the *format* of an email address, but its *deliverability*: a live DNS lookup confirms the domain actually has mail servers before an account is created, rejecting fabricated domains outright.
- New accounts are created in an unverified state. A real verification email is sent (via SMTP, tested with a live Gmail account) containing a one-time link; login is blocked with a clear error until that link is clicked.
- Authenticated sessions use JSON Web Tokens (JWT) with an expiry, issued at login and required on every protected endpoint.
- Role-based access control is enforced at the API layer: only the corresponding author of a publication (or a System Administrator) may edit it; only a project's lead researcher, an Institution Administrator, or a System Administrator may edit a project.

5. API Layer

A FastAPI service exposes the database through a documented, interactive REST API (OpenAPI / Swagger). The API is split between read access, available to any client, and write access, which requires authentication and is subject to validation and ownership rules.

5.1 Read Endpoints

- Listing institutions, researchers, publications, projects, and conferences.
- Retrieving a single researcher's full profile.
- Retrieving a researcher's collaboration network — every distinct co-author they have shared a publication with — which is the data required to power the network visualization described in the project specification.

5.2 Write Endpoints and Validation

- **Registration and login** — with the email and password validation described in Section 4.
- **Researcher profile updates** — partial updates to a researcher's own profile only.
- **Publication creation and updates** — the submitting researcher is automatically recorded as the corresponding author; DOI format and uniqueness are validated; status changes must follow the workflow draft → submitted → published → archived, with any attempt to skip a step rejected.
- **Project creation and updates** — budget cannot be negative; the end date cannot precede the start date; the submitting researcher is recorded as the project lead by default.

6. Entity Relationship Diagram

The diagram below was generated directly from the live schema and shows all fourteen tables with their columns, primary keys, foreign keys, and the cardinality of every relationship.

Scientific Collaboration Network Analyzer — Entity Relationship Diagram

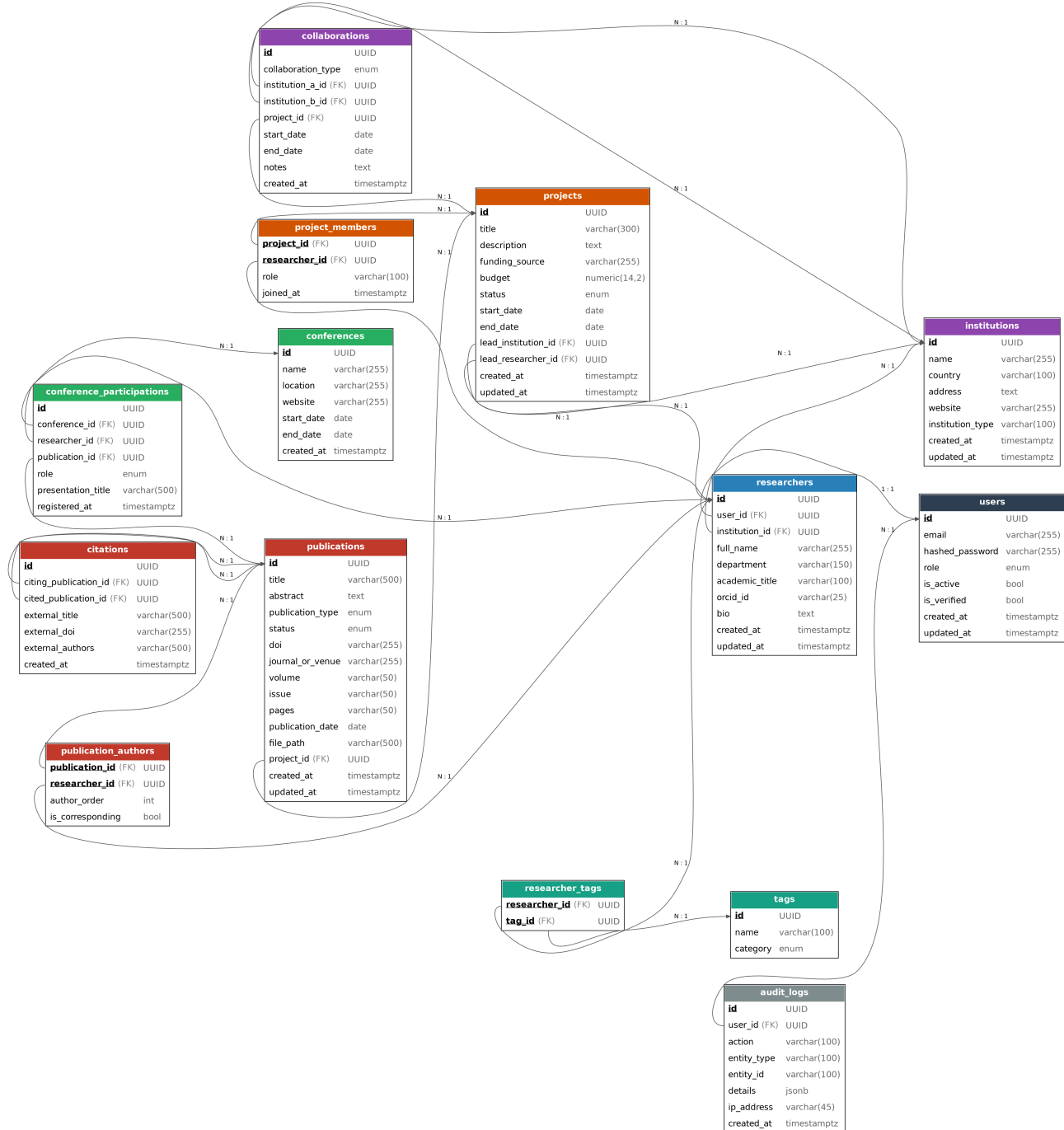


Figure 1. Entity Relationship Diagram — Scientific Collaboration Network Analyzer

7. Testing and Verification

Every feature described in this report was exercised against a live PostgreSQL database rather than left as untested code. The following scenarios were run and confirmed to behave correctly:

- Registration with a real, deliverable email domain succeeds; registration with a fabricated domain is rejected.
- Login is blocked with a 403 error before email verification, and succeeds immediately after the verification link is followed.
- Duplicate email addresses, duplicate DOIs, and duplicate ORCID identifiers are all rejected with clear error messages.
- Weak passwords, malformed emails, and malformed ORCID/DOI values are rejected with field-level validation errors.
- A publication's status cannot be advanced out of order (for example, draft directly to published is rejected).
- A researcher who is not the corresponding author is blocked from editing another researcher's publication.
- Created records were confirmed to persist correctly in PostgreSQL by inspecting the raw table rows directly.

8. Technology Stack

Layer	Technology
Database	PostgreSQL 16
ORM	SQLAlchemy 2.0
Migrations	Alembic
API framework	FastAPI
Authentication	JWT (PyJWT), bcrypt password hashing
Email delivery	SMTP (tested with a live Gmail account)
Containerization	Docker & Docker Compose
Database administration	pgAdmin
Sample data generation	Faker

9. Scope and Ownership

This report covers the database layer specifically: schema design, migrations, authentication, and the validated API endpoints described above. The main application backend logic and the frontend user interface, including the network visualization display, are being developed by other members of the team and are outside the scope of this report. The models and database session defined here are intended to be imported directly by the team's backend, so that the schema is not duplicated or redefined elsewhere.

10. Remaining Work

- **Write endpoints for remaining entities** — conferences, institutions, citations, and conference participation currently support read access only.
- **Finer-grained authorization** — extending role checks (for example, institution-scoped permissions for Institution Administrators) beyond the ownership rules already in place.
- **Dashboards and reporting** — the publication, research, collaboration, and institution reports described in the specification, including PDF/Excel export.
- **Frontend integration** — connecting the team's user interface, including the collaboration network visualization, to the endpoints documented here.
- **Deployment** — packaging the full application for Docker-based deployment as described in the original project outcomes.

11. Conclusion

The database layer of the Scientific Collaboration Network Analyzer is complete, tested, and ready for the rest of the team to build on. It provides a sound relational foundation, real authentication with genuine email verification, and a validated API surface — giving the project a solid base from which the remaining backend logic, reporting, and frontend work can proceed with confidence.